ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DTT
DESARROLLO DE TRANSFERENCIA TECNOLÓGICA



Fecha:	00/00/00

SISTEMAS DE BASES DE DATOS 2

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD 2:
Sistemas Distribuidos

Escuela de Ingenieria de Ciencias Y Sistemas
Facultad de Ingenieria
Universidad de San Carlos de Guatemala

Anuncios Importantes



- **Anuncio 1**
- **Anuncio 2**
- **Anuncio 3**

Agenda

- Definición y Naturaleza
- Arquitecturas Clave
- Ventajas y Desafíos
- Modelos de Consistencia
- Algoritmos de Consenso
- Bases de Datos Distribuidas

COMPETENCIA(S) QUE DESARROLLAREMOS



El estudiante analiza y comprende los fundamentos teóricos y prácticos de los sistemas distribuidos, identificando sus arquitecturas base (Cliente-Servidor, P2P), evaluando los desafíos inherentes (latencia, fallos parciales) y aplicando modelos de consistencia y algoritmos de consenso adecuados para el diseño de soluciones escalables y resilientes en entornos de bases de datos distribuidas.





VALOR DE LA SEMANA

COOPERACIÓN

“Al igual que en un algoritmo de consenso distribuido, un equipo humano es más resiliente y eficaz cuando sus miembros se comunican constantemente, sincronizan sus objetivos y son capaces de cubrir las funciones de un compañero que enfrenta dificultades, logrando que el resultado del grupo sea superior a la suma de sus partes individuales"

Introducción a los Sistemas Distribuidos

Arquitecturas, Consistencia, Consenso y Aplicaciones

Definición de un Sistema Distribuido

Concepto Fundamental:
Un sistema distribuido es una colección de computadoras independientes que aparecen ante los usuarios del sistema como una única computadora coherente.

Componentes Clave:

Nodos: Computadoras individuales con su propio procesador y memoria.
Red: Medio de comunicación que conecta los nodos.
Middleware: Capa de software que gestiona la heterogeneidad y la comunicación entre componentes.

La Ilusión de Imagen Única:
El objetivo final es la transparencia. El usuario no debe saber qué servidor procesa su solicitud, dónde están almacenados los datos o si una réplica ha fallado y ha sido reemplazada.

Características Principales

1. Concurrencia:
Los procesos en diferentes nodos se ejecutan simultáneamente e interactúan entre sí. Esto requiere mecanismos robustos de sincronización y bloqueo para evitar condiciones de carrera.

2. Ausencia de un Reloj Global:
En un sistema centralizado, el reloj del sistema es único. En un sistema distribuido, cada nodo tiene su propio reloj físico. La sincronización perfecta es imposible debido a la variabilidad en los tiempos de transmisión de mensajes, lo que complica la ordenación de eventos (timestamps lógicos vs. físicos).

3. Fallos Independientes:
Si un nodo falla, el sistema total no debería colapsar. La detección de fallos es compleja: ¿el nodo cayó o la red es lenta?

Características Principales

El estándar ISO/RM-ODP define varios tipos de transparencia necesarias:

Acceso: Oculta las diferencias en la representación de datos y mecanismos de invocación.

Ubicación: Oculta dónde reside un objeto o recurso.

Migración: Permite mover recursos sin afectar la operación.

Replicación: Oculta que existen múltiples copias de un recurso para mejorar el rendimiento o la fiabilidad.

Concurrencia: Oculta que varios usuarios compiten por recursos.

Fallas: Oculta el fallo y recuperación de componentes

Arquitecturas Distribuidas - Cliente-Servidor (C-S)

Modelo clásico donde existen roles definidos. El servidor proporciona recursos o servicios, y los clientes inician solicitudes para consumirlos.

Variantes:

2-Capas (2-Tier): Cliente grueso (lógica + UI) y Servidor de base de datos.
3-Capas (3-Tier): Cliente (UI), Servidor de Aplicaciones (Lógica de negocio), Servidor de Datos.
N-Capas (N-Tier): Desglose granular, común en microservicios.

Características:
Centralización de control y seguridad.
Cuello de botella potencial en el servidor.
Escalabilidad vertical limitada, escalabilidad horizontal posible mediante balanceadores de carga.

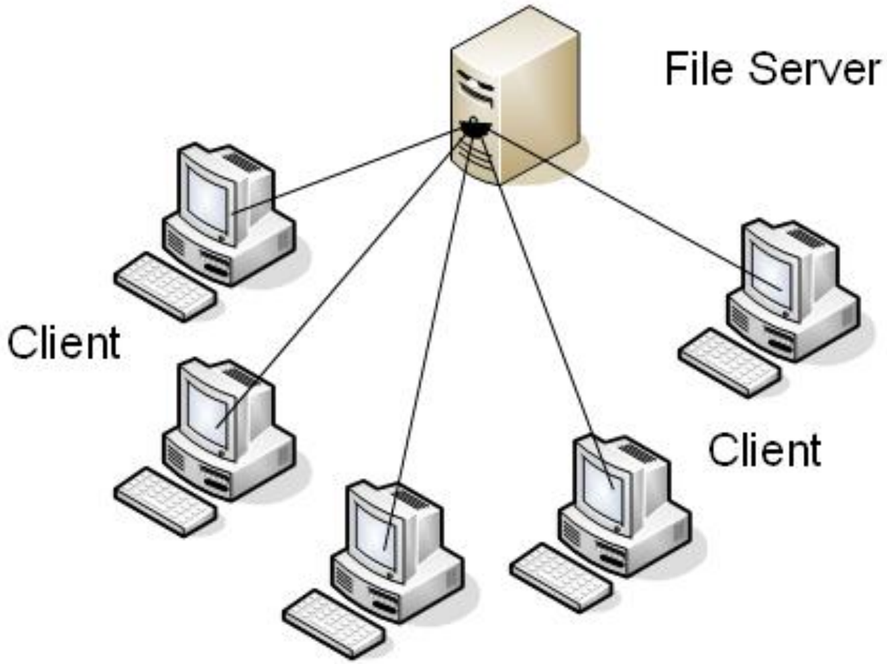
Arquitecturas Distribuidas - Peer-to-Peer (P2P)

No hay distinción fija entre clientes y servidores. Cada nodo (peer) actúa simultáneamente como cliente y servidor, aportando recursos (ancho de banda, almacenamiento, cómputo) al sistema.

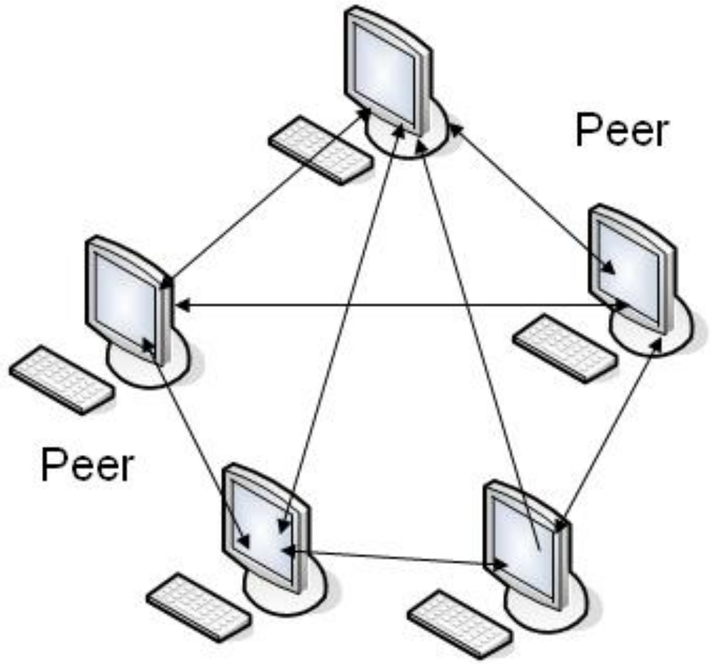
Tipos:

Estructuradas: Usan Tablas Hash Distribuidas (DHT) para una búsqueda eficiente (ej. Chord, Kademlia. Usado en BitTorrent moderno). Determinista.
No Estructuradas: Redes aleatorias o basadas en inundación (flooding) para búsquedas (ej. Gnutella original). Menos eficiente pero más robusta ante alta rotación de nodos (churn).

Usos:
Intercambio de archivos, Blockchains, VoIP (Skype antiguo), Redes de entrega de contenido (CDN híbridas).



Client server mode



Peer to Peer mode

Comparación: C-S vs P2P

Característica	Cliente-Servidor	Peer-to-Peer (P2P)
Control	Centralizado	Descentralizado
Punto de Fallo	Único (el servidor)	Robusto (muchos nodos)
Escalabilidad	Costosa (hardware servidor)	Orgánica (más usuarios = más recursos)
Seguridad	Más fácil de administrar	Difícil (confianza entre pares)
Complejidad	Baja a Media	Alta (gestión de estado distribuido)
Ejemplo	Web server, Email	BitTorrent, Bitcoin

Ventajas de los Sistemas Distribuidos

1. Escalabilidad Horizontal (Scale Out):
Capacidad de agregar más nodos baratos al sistema para aumentar el rendimiento, en lugar de comprar una supercomputadora (Scale Up).

2. Tolerancia a Fallos y Disponibilidad:
La redundancia de datos y servicios permite que el sistema continúe operando, incluso si algunos nodos fallan (Alta Disponibilidad o HA).

3. Baja Latencia:
Al distribuir nodos geográficamente (ej. Edge Computing o CDNs), los datos están más cerca del usuario final.

4. Compartición de Recursos:
Hardware (discos, impresoras) y Software (archivos, bases de datos) compartidos eficientemente.

El Teorema CAP

Formulado por Eric Brewer, establece que un sistema de datos distribuido no puede garantizar simultáneamente más de dos de las siguientes tres propiedades:

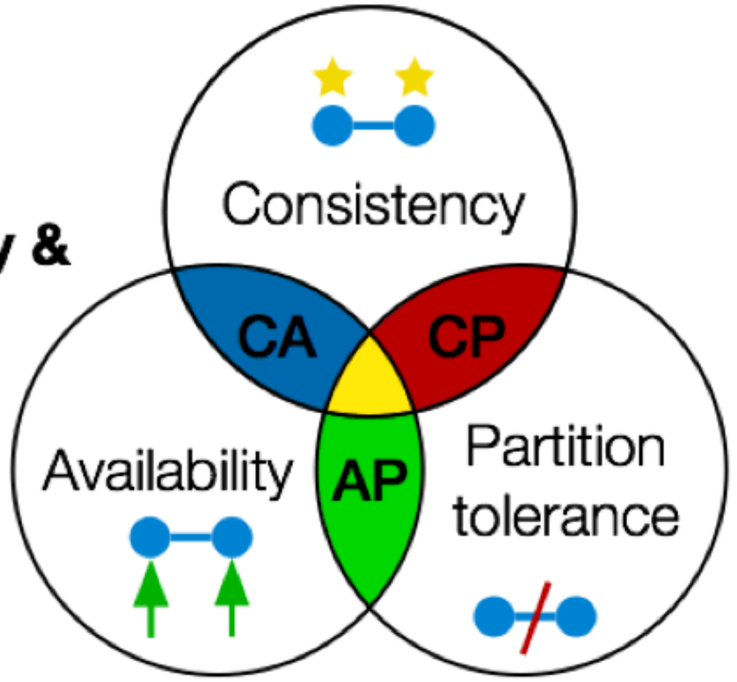
Consistencia (C): Todos los nodos ven los mismos datos al mismo tiempo. (Lectura retorna la última escritura).

Disponibilidad (A): Cada petición recibe una respuesta (sin garantía de que contenga la escritura más reciente), sin importar si algún nodo ha caído.

Tolerancia a la Partición (P): El sistema sigue funcionando a pesar de que la red se rompa y los nodos no puedan comunicarse entre sí.

En la práctica: En sistemas distribuidos grandes, la partición de red es inevitable (P debe existir), por lo que se debe elegir entre CP (Consistencia fuerte, baja disponibilidad) o AP (Alta disponibilidad, consistencia eventual).

El Teorema CAP



Consistency & Availability
- MySQL
- PostgreSQL

Consistency & Partition tolerance
- HBase
- MongoDB
- Redis
- Memcache

Availability & Partition tolerance
- Riak
- Cassandra
- CouchDB
- DynamoDB

Modelos de Consistencia - Fuerte vs. Débil

Consistencia Fuerte (Strict/Linearizability):

Cualquier lectura retorna el valor de la escritura más reciente.
Requiere sincronización global.
Impacto negativo alto en latencia y disponibilidad.

Ejemplo: Transacciones bancarias críticas (Base de datos SQL tradicional en configuración estricta).

Consistencia Débil/Eventual:

Si no hay nuevas actualizaciones, eventualmente todos los accesos retornarán el último valor actualizado.
Existe una "ventana de inconsistencia".
Permite alta disponibilidad y baja latencia.

Ejemplo: Contadores de "Likes" en redes sociales, DNS.

Algoritmos de Consenso - El Problema de los Generales Bizantinos

El Problema:

¿Cómo logran un grupo de nodos ponerse de acuerdo en un valor (ej. quién es el líder, o si se commitea una transacción) cuando algunos nodos pueden fallar o la red es poco fiable?

Requisitos del Consenso:

Terminación: Eventualmente cada proceso correcto decide un valor.
Acuerdo: Todos los procesos correctos deciden el mismo valor.
Integridad: Si todos los procesos correctos proponen el mismo valor v, entonces todos deciden

Algoritmos de Consenso - Paxos

Paxos (Leslie Lamport):

Es el "estándar de oro" teórico para el consenso en redes no fiables con fallos de parada (no maliciosos).

Roles: Proposers (proponen valores), Acceptors (votan), Learners (aprenden el resultado).

Funcionamiento: Fase de preparación (promesa) y Fase de aceptación.

Desventaja: Es notoriamente difícil de entender y de implementar correctamente.

Algoritmos de Consenso - Raft

Raft (Diego Ongaro y John Ousterhout):

Diseñado específicamente para ser comprensible. Equivalente a Paxos en tolerancia a fallos.

Enfoque: Se basa en la elección de un Líder Fuerte.

Mecanismo:

- Elección de Líder: Los nodos votan. Solo el líder acepta peticiones de clientes.
- Replicación de Logs: El líder recibe el comando, lo añade a su log, lo envía a los seguidores. Una vez que la mayoría confirma, el líder aplica (commit) y responde al cliente.

Usado en: Kubernetes (etcd), MongoDB, Kafka (versiones modernas).

Casos de Uso - Bases de Datos Distribuidas

1. NoSQL (Not Only SQL):

Diseñadas para escalar horizontalmente y manejar grandes volúmenes de datos no estructurados.

- **Documentales (MongoDB):** JSON flexible, para catálogos y CMS.
- **Columnar (Cassandra/HBase):** Escrituras masivas, series temporales, big data.
- **Clave-Valor (Redis/DynamoDB):** Caché, sesiones de usuario, carritos de compra.

2. NewSQL:

Bases de datos que intentan mantener las garantías ACID de SQL tradicional pero con la escalabilidad de NoSQL (Ej. Google Spanner, CockroachDB).

Casos de Uso - Aplicaciones Reales

Facebook / Meta:

- Usa Cassandra (o variantes) para la búsqueda en la bandeja de entrada (necesidad de alta escritura).

- Usa Memcached (distribuido) para acelerar la carga de perfiles y reducir carga en la BD.

Google:

- **Google File System (GFS) / Colossus**: Sistema de archivos distribuido para almacenar índices de búsqueda y videos de YouTube.

- **MapReduce:** Modelo de programación distribuida para procesar esos datos.

Blockchain (Bitcoin/Ethereum):

- Base de datos distribuida peer-to-peer con Consenso Proof of Work (PoW) o Proof of Stake (PoS) para validar transacciones sin autoridad central.

CONCEPTOS CLAVE APRENDIDOS

Transparencia: La clave de un buen sistema distribuido es ocultar la complejidad de la red y los fallos al usuario.

Trade-offs (Compromisos): No existe el sistema perfecto. Según el Teorema CAP, siempre debemos sacrificar consistencia o disponibilidad ante particiones.

Escalabilidad vs. Rendimiento: Escalar (añadir nodos) tiene un costo de coordinación que puede reducir el rendimiento individual de cada nodo, pero aumenta la capacidad total.

Consenso: Raft y Paxos son fundamentales para mantener la integridad de los datos replicados en entornos con fallos.

REFERENCIAS

https://www.geeksforgeeks.org/operating-systems/types-of-distributed-system/

https://highlights.cis.upenn.edu/distributed-systems-networks-and-operating-systems/



Ejemplo práctico



Practiquemos lo aprendido

ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DTT
DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Nombre de la actividad:	Actividad Semana 6
Cantidad de participantes:	---
Doy fe que esta actividad está planificada en dtt (Sí/No):	Si

Hora de inicio:	---
Hora de fin:	---
Duración (min):	---

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

---	---	---
---	---	---
---	---	---
---	---	---
---	---	---
---	---	---

Recursos



DUDAS ¿?



¡GRACIAS POR SU ATENCIÓN!

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA SEMANA